Gaussian Splatting for Physical Inverse Problems:

3D Gaussians for Inverse Scattering and 4D Gaussians for Fluid Dynamics

Thesis I Report

Muhammad Hussain Qureshi
[Roll number, e.g. MSxx26xxx]

Advisor: [Advisor name]

Faculty of Sciences
Information Technology University (ITU), Lahore

December 2026

Abstract

3D Gaussian Splatting represents a scene as a set of anisotropic Gaussians and has become a leading representation for photographic novel-view synthesis; its dynamic extension, 4D Gaussian Splatting, adds a deformation network over time. Both are designed for posed RGB images. This thesis asks whether the same representation is useful for *scientific inverse problems*, where an unknown physical field must be recovered from indirect, incomplete and noisy measurements produced by a known physics model. Using datasets and forward models from the InverseBench benchmark, we study two problems: (i) linear inverse scattering, where the permittivity of cells is recovered from scattered electromagnetic fields measured by 360, 180 or 60 receivers, in 2D and on 3D cell volumes; and (ii) the 2D incompressible Navier–Stokes equations, where an evolving vorticity field is reconstructed from dense or sparse observations over time with a 4D (space–time) Gaussian representation whose deformation is modelled either by a multilayer perceptron, as in 4D Gaussian Splatting, or by a Mamba selective state-space model. We replace the camera rasteriser with the physics operator, render Gaussians on a grid with a local-window evaluation, and train end-to-end by gradient descent with 3DGS-style densification and pruning.

The proof of concept shows that (1) on smooth 3D cell volumes, Gaussians are markedly more robust than a voxel grid when measurements are scarce (40.9 vs. 37.3 dB PSNR at 60 receivers); (2) on textured 2D cells, extra Gaussians can grow "ghost" copies of the object in regions the measurements cannot see (the operator's null space), which a simple sparsity term removes, after which Gaussians exceed the pixel baseline in structural similarity at 60 receivers (SSIM 0.940 vs. 0.878) while the pixel grid keeps higher PSNR; and (3) for fluid flow, a deformation model is essential (+10 dB over static Gaussians), while Mamba fits observed frames better than the MLP but does not yet generalise better to unobserved time steps. These results establish feasibility and point to the open problems addressed in Thesis II.

Contents

Abstract **i**

1 Introduction **1**
1.1 Motivation 1
1.2 Problem statement (summary) 2
1.3 Contributions of Thesis I 2
1.4 Outline 2

2 Background **3**
2.1 Inverse problems 3
2.2 Linear inverse scattering 3
2.3 Incompressible Navier–Stokes flow 4
2.4 Neural fields and Gaussian splatting 4
2.5 State-space sequence models and Mamba 5

3 Literature Review **6**
3.1 Priors for scientific inverse problems 6
3.2 Explicit Gaussian representations 7
3.3 Gaussians and neural fields in scientific imaging 7
3.4 Dynamic representations and sequence models 7
3.5 Synthesis and research gap 8

4 Problem Definition and Research Scope **9**
4.1 Problem definition 9
4.2 Research questions 9
4.3 Scope 10
4.4 Assumptions 10

5 Proposed Methodology and Pipeline Architecture **11**
5.1 Overview 11
5.2 Gaussian field representation 11
5.3 Physics operators 11

5.4 Training and adaptive density control . . . 12
5.5 Sparsity regulariser . . . 12
5.6 4D representation of fluid flow . . . 13
5.7 Baselines . . . 13
5.8 Implementation . . . 13

6 Mathematical Formulation **14**
6.1 Gaussian field . . . 14
6.2 Scattering forward model . . . 14
6.3 Objective for scattering . . . 15
6.4 Adaptive density control . . . 15
6.5 4D representation . . . 15
6.6 Mamba deformation . . . 16
6.7 Evaluation metrics . . . 16

7 Experimental Setup **17**
7.1 Data . . . 17
7.2 Forward models and noise . . . 17
7.3 Hyper-parameters . . . 18
7.4 Hardware and software . . . 18
7.5 Protocol . . . 18

8 Initial Baseline Results **19**
8.1 Representation check . . . 19
8.2 2D inverse scattering on InverseBench cells . . . 19
8.3 3D inverse scattering . . . 19
8.4 4D Gaussians for Navier–Stokes . . . 20
8.5 Answers to the research questions so far . . . 21

9 Discussion, Limitations and Plan for Thesis II **23**
9.1 Discussion . . . 23
9.2 Limitations . . . 23
9.3 Plan for Thesis II . . . 24
9.4 Conclusion . . . 24

A Reproducibility **29**

List of Figures

5.1 Pipeline. The camera rasteriser of 3DGS is replaced by rendering on a grid followed by the physics operator. 12

8.1 Cell 0. Top: aggressive densification without regularisation produces a ghost copy beside the cell. Bottom: with the sparsity term ($\lambda = 0.3$) the ghost is gone. Columns: ground truth, then Gaussian and pixel reconstructions at 360, 180 and 60 receivers. 20

8.2 Middle slice of synthetic volume 0: ground truth, direct fit, then Gaussian and voxel reconstructions at 360, 180 and 60 receivers. 21

8.3 Vorticity of sequence 0 at five times (held-out frames marked). Rows: ground truth, then MLP, Mamba and static models with 100% and 10% of pixels observed. 22

List of Tables

3.1 Representative related work. “Meas.” is the kind of measurement used for supervision. 8

7.1 Default settings (overridable from the command line; every run stores its exact settings). 18

8.1 2D inverse scattering, InverseBench cells (PSNR dB / SSIM, mean of 3). . 20
8.2 3D inverse scattering, $32 \times 128 \times 128$ volumes (PSNR dB / SSIM, mean of 3). 20
8.3 4D Navier–Stokes, PSNR (dB) on held-out frames; observed-frame PSNR in brackets (mean of 3 sequences). 21

Chapter 1

Introduction

1.1 Motivation

Many measurements in science do not show the quantity of interest directly. An optical tomography system records how light is scattered by a cell, not the cell's refractive index; a fluid experiment samples the flow at a few points and times, not the full velocity or vorticity field. Recovering the hidden quantity from such indirect, incomplete and noisy data is an *inverse problem*. Inverse problems are typically ill-posed: many different unknowns explain the measurements equally well, so the representation of the unknown and the prior knowledge built into it decide which solution is found.

In computer vision, 3D Gaussian Splatting (3DGS) [9] has recently changed how scenes are represented. Instead of a pixel or voxel grid, or a neural network evaluated at every point [16], a scene is a set of anisotropic Gaussians with learnable positions, shapes and amplitudes. The representation is compact, adapts its resolution to the content through densification and pruning, and is differentiable, so it can be fitted to measurements by gradient descent. Its dynamic extension, 4D Gaussian Splatting [27], keeps one canonical set of Gaussians and learns a deformation network that moves them over time.

Gaussians are also a natural basis for many physical fields: cell nuclei are smooth blobs, and vortices in a fluid are localised, smooth structures that are carried along by the flow. This suggests that a Gaussian representation could act as a useful structural prior for scientific inverse problems. Whether it does, and under which conditions, has not been studied systematically: existing Gaussian methods for scientific imaging target specific modalities such as X-ray computed tomography [2, 30], while the recent InverseBench benchmark [32] evaluates diffusion-model priors, not explicit Gaussian representations.

1.2 Problem statement (summary)

This thesis studies 3D and 4D Gaussian representations as the unknown in physics-based inverse problems. Given a known forward model $\mathcal{A}$ that maps a physical field to measurements, and measurements $\boldsymbol{y} = \mathcal{A}(f) + \boldsymbol{n}$, we represent f by Gaussians and fit their parameters so that the simulated measurements match $\boldsymbol{y}$. We ask when this representation helps compared with a plain grid of unknowns, how it fails, and how its motion over time should be modelled. The full problem definition and scope are given in Chapter 4.

1.3 Contributions of Thesis I

1. **Dataset analysis.** A study of the five InverseBench problems with respect to their suitability for 3D and 4D Gaussian representations, leading to the choice of linear inverse scattering (3D) and Navier–Stokes (4D).

2. **A physics-driven Gaussian pipeline.** A single implementation that renders 2D/3D Gaussians on a grid with a local-window evaluation, couples them to the InverseBench scattering operator (verified to match the benchmark implementation to within 3×10^{-7} relative error) and to observations of the Navier–Stokes solution, and trains them with densification, pruning and an optional sparsity term.

3. **4D Gaussians with a state-space motion model.** A 4D (2D + time) Gaussian representation whose deformation is modelled either by an MLP, as in 4D Gaussian Splatting, or by a Mamba selective state-space model [7] that treats each Gaussian's trajectory as a sequence.

4. **Initial results and a failure analysis.** Baseline comparisons against pixel/voxel optimisation with identical physics, and the identification of a "ghost" failure mode caused by the operator's null space, with a sparsity-based remedy.

5. **Open, reproducible code.** A public repository with a Python package, experiment configurations, notebooks, unit tests and all results so far (Appendix A).

1.4 Outline

Chapter 2 introduces inverse problems, the two physical models and Gaussian splatting. Chapter 3 reviews related work. Chapter 4 defines the problem, research questions and scope. Chapter 5 describes the pipeline and Chapter 6 its mathematical formulation. Chapter 7 gives the experimental setup and Chapter 8 the initial results. Chapter 9 discusses limitations and the plan for Thesis II.

Chapter 2

Background

2.1 Inverse problems

An inverse problem asks for an unknown f from measurements

$$\boldsymbol{y} = \mathcal{A}(f) + \boldsymbol{n}, \tag{2.1}$$

where $\mathcal{A}$ is a known forward model (here, a physics simulator) and $\boldsymbol{n}$ is noise. The problem is *ill-posed* when a solution does not exist, is not unique, or does not depend continuously on the data. In the problems studied here there are far fewer measurements than unknowns, so $\mathcal{A}$ has a large null space: many fields produce exactly the same measurements. A solution is therefore chosen by a prior, either explicit, as a regularisation term such as total variation [21] or sparsity, or implicit, through the *parametrisation* of the unknown. A convolutional network trained on a single image acts as such an implicit prior [24]; diffusion models trained on many examples act as a learned prior and are the subject of InverseBench [4, 32]. In this thesis the prior is the Gaussian representation itself, optionally combined with a sparsity term.

2.2 Linear inverse scattering

In optical diffraction tomography an object is illuminated by a known incident field and the scattered field is recorded by receivers around it [1]. For a weakly scattering object the total field inside the object can be approximated by the incident field (the first Born approximation), and the scattered field becomes a *linear* function of the object's permittivity contrast $f(\boldsymbol{x})$:

$$u_t^{\mathrm{sc}}(\boldsymbol{r}) = k_b^2 \int_\Omega G(\boldsymbol{r},\boldsymbol{x})\, f(\boldsymbol{x})\, u_t^{\mathrm{in}}(\boldsymbol{x})\, d\boldsymbol{x}, \qquad G(\boldsymbol{r},\boldsymbol{x}) = \tfrac{i}{4} H_0^{(1)}(k_b \|\boldsymbol{r}-\boldsymbol{x}\|)\,, \tag{2.2}$$

where t indexes the transmitter, $\boldsymbol{r}$ a receiver position, k_b the background wavenumber and $H_0^{(1)}$ the Hankel function of the first kind (the 2D Green's function). Multiple-scattering models drop the Born approximation at a higher cost [22]. InverseBench uses the linear model with 20 transmitters on a circle and 360, 180 or 60 receivers on the same circle, a 128×128 grid of a $0.18 \times 0.18\,\mathrm{m}$ domain, and cell images simulated with CytoPacq [26].

2.3 Incompressible Navier–Stokes flow

The 2D incompressible Navier–Stokes equations in vorticity form on the periodic domain $[0, 2\pi]^2$ are

$$\partial_t \omega + \boldsymbol{u} \cdot \nabla \omega = \frac{1}{Re} \Delta \omega + g, \qquad \boldsymbol{u} = (\partial_y \psi,\, -\partial_x \psi), \qquad -\Delta \psi = \omega, \tag{2.3}$$

with vorticity ω, velocity $\boldsymbol{u}$, stream function ψ, Reynolds number Re and forcing g. Vorticity is transported by the flow and diffused by viscosity, so its structures move and deform smoothly over time. InverseBench uses $Re = 200$, forcing $g = -4\cos(4y)$ and a pseudo-spectral solver; this problem is also a standard benchmark for neural PDE solvers [13].

2.4 Neural fields and Gaussian splatting

Neural radiance fields. NeRF [16] represents a scene by an MLP that maps a 3D position and viewing direction to colour and density and renders images by volume rendering along camera rays. Fourier features of the input coordinates allow MLPs to represent high frequencies [23]. Grid- and hash-based encodings make such fields much faster [17].

3D Gaussian Splatting. 3DGS [9] replaces the MLP by an explicit set of anisotropic 3D Gaussians, each with a mean, a covariance factorised into a rotation and scales, an opacity and a colour. Gaussians are projected ("splatted") onto the image plane and alpha-blended; all parameters are optimised by gradient descent. *Adaptive density control* clones or splits Gaussians where reconstruction gradients are large and prunes transparent ones, so the representation spends its capacity where the scene needs it.

Dynamic scenes. D-NeRF [18] introduced a canonical space plus a deformation network for dynamic NeRFs. 4D Gaussian Splatting [27] applies the same idea to Gaussians, predicting per-time offsets of position, scale and rotation from a spatio-temporal feature encoding (HexPlane [3]). Alternatives include per-frame Gaussians tracked over time [15] and native 4D Gaussians [28].

2.5 State-space sequence models and Mamba

Structured state-space models (S4) [8] model sequences with a linear recurrence $\boldsymbol{h}_t = \bar{\mathbf{A}}\boldsymbol{h}_{t-1} + \bar{\mathbf{B}}x_t$, $y_t = \mathbf{C}\boldsymbol{h}_t$, obtained by discretising a continuous-time system. Mamba [7] makes the step size Δ and the matrices $\mathbf{B}$, $\mathbf{C}$ functions of the input (*selection*), so the model can decide what to keep in its state, and computes the recurrence with a hardware-efficient scan in time linear in the sequence length. Mamba has been applied to vision backbones [33]. Here it is used to model how each Gaussian moves through time.

Chapter 3

Literature Review

This chapter reviews four lines of work that meet in this thesis: (1) learned priors for scientific inverse problems, (2) explicit Gaussian representations, (3) Gaussian and neural-field methods for scientific and medical imaging, and (4) dynamic representations and sequence models. It closes with the gap this thesis addresses.

3.1 Priors for scientific inverse problems

Classical solvers combine a data-fidelity term with hand-crafted regularisers such as total variation [21]. Deep learning added two families of priors. *Untrained* priors use the structure of a network as the regulariser: Deep Image Prior [24] fits a randomly initialised convolutional network to a single corrupted image and stops early. *Learned* priors are trained on many examples; diffusion models are currently the most general, and Diffusion Posterior Sampling [4] guides the reverse diffusion with the gradient of the data likelihood, so the same prior can serve many forward models.

InverseBench [32] evaluates 14 such plug-and-play diffusion-prior algorithms on five scientific problems: linear inverse scattering, compressed-sensing MRI on fastMRI [29], black hole imaging, full waveform inversion on OpenFWI [5], and the 2D Navier–Stokes equations. Its findings that motivate this thesis are that (i) scientific forward models differ strongly from image-restoration operators (complex-valued, non-linear, expensive, or without gradients), (ii) diffusion priors help most when measurements are sparse, and (iii) they can fail when outputs violate physical constraints. All its methods represent the unknown on a pixel grid. Physics-informed neural networks [20] and neural operators such as FNO [13] are complementary: they learn solution operators of PDEs rather than priors over unknowns.

3.2 Explicit Gaussian representations

3DGS [9] showed that an explicit, adaptive set of anisotropic Gaussians can match or exceed neural fields [16, 17] in quality while rendering in real time. Its key ingredients are a differentiable rasteriser, a covariance parametrisation by rotation and scale that stays positive semi-definite, and adaptive density control. GaussianImage [31] transfers the idea to 2D: an image is represented by 2D Gaussians and rendered by accumulated summation, achieving fast decoding and compression. Our 2D and 3D representations follow the same principle, but are evaluated on a regular grid and supervised through a physics operator instead of pixels.

3.3 Gaussians and neural fields in scientific imaging

Tomography. X-Gaussian [2] adapts Gaussian splatting to X-ray projection imaging with a radiative Gaussian model, and R^2-Gaussian [30] rectifies the integration bias of splatting for tomographic volume reconstruction, showing that Gaussians can serve as a voxel-free representation of attenuation volumes. Both rely on ray-based projection, the natural analogue of camera rendering; scattering measurements are instead a wave-based integral over the whole object.

Astronomy. BH-NeRF [11] reconstructs 3D emission around a black hole over time with a neural field constrained by gravitational lensing and orbital dynamics, an example of physics replacing multi-view geometry.

Microscopy data. Simulated cell images with exact ground truth are produced by CytoPacq and related generators [26]; the Broad Bioimage Benchmark Collection [14] distributes such synthetic 3D nucleus datasets (BBBC024), which we use as real 3D cell shapes.

3.4 Dynamic representations and sequence models

D-NeRF [18] and 4D Gaussian Splatting [27] model motion with a canonical representation and a deformation field queried independently at each time; HexPlane [3] and K-Planes [6] factorise space–time features into planes. Dynamic 3D Gaussians [15] track persistent Gaussians frame by frame, and native 4D Gaussians [28] slice 4D primitives in time. Language-embedded variants (LangSplat [19], 4D LangSplat [12]) attach semantic features to such Gaussians, the starting point of this thesis' original direction. None of these treats a Gaussian's trajectory as a sequence. State-space models [7, 8] are efficient sequence

Table 3.1: Representative related work. "Meas." is the kind of measurement used for supervision.

Work	Representation	Meas.	Relation to this thesis
NeRF [16]	MLP field	RGB images	implicit field; rendering by rays
3DGS [9]	3D Gaussians	RGB images	our representation, without cameras
4DGS [27]	Gaussians + deformation	video	our 4D design (MLP deformation)
GaussianImage [31]	2D Gaussians	pixels	2D Gaussian rendering on a grid
X-Gaussian [2], R^2-Gaussian [30]	3D Gaussians	X-ray projections	Gaussians for physics, ray-based
BH-NeRF [11]	neural field	radio interferometry	physics replaces multi-view geometry
DPS [4], InverseBench [32]	pixel grid + diffusion prior	physics operators	same problems, learned prior
Mamba [7]	selective SSM	sequences	our motion model over time
This thesis	2D/3D/4D Gaussians	scattering; sparse flow samples	Gaussians + physics; Mamba motion

models that could do exactly that, and have been adopted in vision [33]; their use as a motion model for 4D Gaussians is, to our knowledge, unexplored.

3.5 Synthesis and research gap

Table 3.1 summarises the landscape. Diffusion-prior methods are general but represent the unknown on a grid and need large training sets; Gaussian methods are compact and adaptive but have been demonstrated mainly with camera- or ray-based measurements. The gap addressed here is the use of 3D/4D Gaussian representations as the unknown in *wave-based and PDE-based scientific inverse problems*, the analysis of their failure modes under severe ill-posedness, and sequence-model motion priors for their dynamics.

Chapter 4

Problem Definition and Research Scope

4.1 Problem definition

Let $f : \Omega \to \mathbb{R}$ be an unknown physical field on a domain $\Omega \subset \mathbb{R}^d$ (permittivity contrast of a cell, $d \in \{2, 3\}$), or a time-dependent field $\omega : \Omega \times [0, T] \to \mathbb{R}$ (vorticity of a flow, $d = 2$). A known forward model $\mathcal{A}$ produces measurements $\boldsymbol{y} = \mathcal{A}(f) + \boldsymbol{n}$ with $\dim \boldsymbol{y} \ll \dim f$. We represent the unknown by a set of N Gaussians with parameters $\Theta = \{\theta_i\}_{i=1}^{N}$ (and, for dynamic fields, a deformation network with weights ϕ) and solve

$$\hat{\Theta} = \arg\min_{\Theta} \; \mathcal{D}\big(\mathcal{A}(f_\Theta), \boldsymbol{y}\big) + \mathcal{R}(\Theta), \tag{4.1}$$

where $\mathcal{D}$ measures data misfit and $\mathcal{R}$ is an optional regulariser. The ground truth is never used for fitting; it is used only to evaluate the reconstruction.

4.2 Research questions

RQ1 Can 2D/3D Gaussian representations recover physical fields from scattering measurements, and how do they compare with a pixel/voxel grid optimised with the same physics as the number of measurements decreases?

RQ2 What failure modes arise when an adaptive Gaussian representation meets an operator with a large null space, and how can they be prevented?

RQ3 Can a 4D Gaussian representation (canonical Gaussians plus deformation) reconstruct an evolving fluid field from dense and sparse observations, including at time steps that were never observed?

RQ4 Does a selective state-space model (Mamba) over each Gaussian's trajectory model the dynamics better than the per-time MLP deformation of 4D Gaussian Splatting?

4.3 Scope

In scope (Thesis I).

- Linear inverse scattering with the InverseBench forward model: 2D cells (InverseBench test set) and 3D cell volumes (synthetic HL60-like nuclei and CytoPacq/BBBC024 nuclei), with 360, 180 and 60 receivers and the benchmark noise level.
- 3D physics approximated by applying the 2D operator to every z-slice (*slice-wise Born model*).
- 2D Navier–Stokes ($Re = 200$) as a 2D + time ("4D") problem: sequences of 21 frames, observations of all or 10% of the pixels per frame, every fourth frame held out.
- Baselines: direct optimisation of a pixel/voxel grid with the same data term (and the same sparsity term).
- Metrics: PSNR, SSIM, relative ℓ_2 error, number of Gaussians, run time.

Out of scope (deferred to Thesis II).

- A full 3D scattering model with sensors on a sphere, and non-linear (multiple-scattering) physics.
- Recovering the Navier–Stokes *initial condition* through the solver, as posed in InverseBench.
- Exhaustive benchmarking against the diffusion-prior methods of InverseBench; training learned priors.
- Real (non-simulated) measurement data, and the remaining InverseBench problems.

4.4 Assumptions

The forward models are known exactly and are differentiable; measurement noise is additive Gaussian at the benchmark level ($\sigma = 10^{-4}$ for scattering); permittivity contrast is non-negative and bounded in $[0, 1]$; vorticity is periodic in space.

Chapter 5

Proposed Methodology and Pipeline Architecture

5.1 Overview

The same training loop serves every task (Figure 5.1). A set of Gaussians is rendered on the grid of the unknown; a physics operator turns the rendered field into simulated measurements; the misfit to the real measurements, plus an optional sparsity term, is back-propagated to all Gaussian parameters; and an adaptive density control step periodically adds and removes Gaussians. For dynamic fields a deformation network maps the canonical Gaussians to their state at each time step before rendering.

5.2 Gaussian field representation

Each Gaussian has a centre, per-axis scales, an orientation and an amplitude; the field at a grid point is the sum of all Gaussians evaluated there (Section 6.1). Because a Gaussian is negligible beyond about three standard deviations, each Gaussian is evaluated only in a window of $\pm h$ cells around its centre and accumulated into the grid with a scatter-add, so the cost is $O(N(2h+1)^d)$ and independent of the grid size; scales are clamped to $h/3$ so that the window always covers $\pm 3\sigma$. For permittivity the amplitude is kept non-negative with a softplus; for vorticity it is signed and the window wraps around the periodic domain. All parameters of one scene form a single $N \times P$ tensor ($P = 6$ in 2D, $P = 11$ in 3D), which is convenient for sequence models.

5.3 Physics operators

Scattering. We re-implement the InverseBench linear scattering operator as two complex matrix products in single precision on the GPU and verify at start-up that it matches

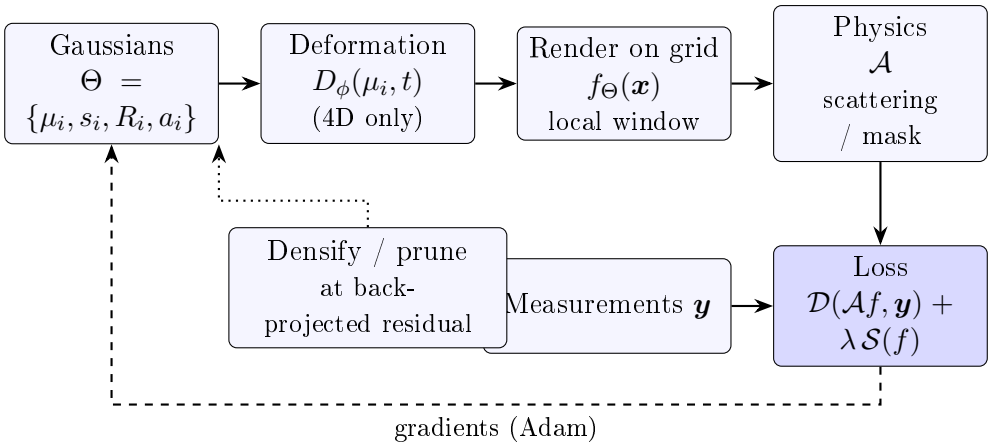


Figure 5.1: Pipeline. The camera rasteriser of 3DGS is replaced by rendering on a grid followed by the physics operator.

the benchmark implementation to within a relative error of 3×10^{-7}. For 3D volumes, the operator is applied to every z-slice and the losses are summed.

Fluid observations. For Navier–Stokes the measurement operator selects the observed pixels of each frame (a random mask with 100% or 10% coverage); frames that are held out for evaluation are never observed.

5.4 Training and adaptive density control

All parameters are optimised with Adam [10] and a cosine learning-rate schedule. For scattering, Gaussians are initialised uniformly at random, with no knowledge of the object. Every 250 iterations, during the first 60% of training, new Gaussians are inserted at the grid points with the largest magnitude of the *back-projected residual* $\operatorname{Re}\mathcal{A}^{\mathsf{H}}(\boldsymbol{y} - \mathcal{A}f)$, i.e. where the measurements indicate missing structure, and Gaussians whose amplitude has fallen below 10^{-3} are pruned. This replaces the image-space gradient criterion of 3DGS, which is unavailable without pixel supervision.

5.5 Sparsity regulariser

Early experiments showed that aggressive densification can place mass in regions to which the measurements are blind (Section 8.2). A sparsity term on the total intensity of the rendered field penalises mass the data does not require. Because the field is non-negative, the mean of the field equals its ℓ_1 norm divided by the grid size.

5.6 4D representation of fluid flow

A canonical set of 2D Gaussians is initialised from the observed pixels of the first frame. A deformation network maps each Gaussian's canonical centre and the time t to offsets of all its parameters; positions move in pixel units. Three deformation models are compared:

- **MLP** (as in 4D Gaussian Splatting): Fourier features of (μ_i, t) fed to a three-layer MLP, evaluated independently for every Gaussian and time.
- **Mamba**: for each Gaussian the inputs at all T time steps form a sequence processed by two Mamba blocks, so the offset at time t depends on the Gaussian's earlier states (causal).
- **Bidirectional Mamba**: a second Mamba stack reads the sequence backwards and the two are summed, so every time step sees both past and future.

The output layer is initialised to zero so training starts from the canonical configuration. An optional temporal smoothness term penalises differences between offsets at consecutive times, including unobserved times. A *static* model without deformation serves as an ablation.

5.7 Baselines

The baseline for scattering optimises every pixel (2D) or voxel (3D) directly with the same data term, starting from zero and projected to $[0, 1]$ after each step; optionally it uses the same sparsity term. Only the representation differs between the two methods. For fluids, the static-Gaussian model is the baseline.

5.8 Implementation

The pipeline is a Python/PyTorch package (`thesis_gs`) with modules for the Gaussian field, physics, volume loading, scattering experiments, Navier–Stokes data, deformation networks and 4D training, a command-line interface, experiment configurations and unit tests (Appendix A). Mamba is implemented in pure PyTorch with an explicit selective scan, which is sufficient for sequences of 21 steps and runs on any GPU.

Chapter 6

Mathematical Formulation

6.1 Gaussian field

A field on a grid with points $\boldsymbol{x} \in \Omega \subset \mathbb{R}^d$ is represented by N Gaussians,

$$f_\Theta(\boldsymbol{x}) = \sum_{i=1}^{N} a_i \exp\Big(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i)\Big), \qquad \boldsymbol{\Sigma}_i = \mathbf{R}_i \operatorname{diag}(\boldsymbol{s}_i)^2 \mathbf{R}_i^\top, \tag{6.1}$$

with centre $\boldsymbol{\mu}_i \in \mathbb{R}^d$, scales $\boldsymbol{s}_i = \exp(\boldsymbol{\ell}_i)$ clamped to $[s_{\min}, h/3]$, and rotation $\mathbf{R}_i$: in 2D $\mathbf{R}(\vartheta_i) = \left(\begin{smallmatrix} \cos\vartheta_i & -\sin\vartheta_i \\ \sin\vartheta_i & \cos\vartheta_i \end{smallmatrix}\right)$, in 3D the rotation matrix of the normalised quaternion $\boldsymbol{q}_i/\|\boldsymbol{q}_i\|$. The factorisation keeps $\boldsymbol{\Sigma}_i$ symmetric positive definite for any parameter values [9]. The amplitude is $a_i = \operatorname{softplus}(\alpha_i) \geq 0$ for permittivity and $a_i = \alpha_i \in \mathbb{R}$ for vorticity. The parameters per Gaussian are $\theta_i = (\boldsymbol{\mu}_i, \boldsymbol{\ell}_i, \vartheta_i \text{ or } \boldsymbol{q}_i, \alpha_i)$. In practice the sum in (6.1) runs only over Gaussians whose window $\|\boldsymbol{x} - \operatorname{round}(\boldsymbol{\mu}_i)\|_\infty \leq h$ contains $\boldsymbol{x}$.

6.2 Scattering forward model

On a grid of P points with spacing $\Delta x\, \Delta y$, the linear (Born) scattering operator of the benchmark maps a contrast image $\boldsymbol{f} \in \mathbb{R}^P$ to measurements for T transmitters and M receivers,

$$y_{t,r} = \Delta x\, \Delta y \sum_{p=1}^{P} f_p\, \overline{u_t^{\text{in}}(\boldsymbol{x}_p)}\, g_r(\boldsymbol{x}_p), \qquad g_r(\boldsymbol{x}) = k_b^2\, \tfrac{i}{4} H_0^{(1)}\big(k_b \|\boldsymbol{r}_r - \boldsymbol{x}\|\big), \tag{6.2}$$

written compactly as $\boldsymbol{y} = \mathbf{A}\boldsymbol{f} + \boldsymbol{n}$ with $\mathbf{A} \in \mathbb{C}^{TM \times P}$ and complex Gaussian noise of standard deviation $\sigma = 10^{-4}$. Here $T = 20$, $M \in \{360, 180, 60\}$ and $P = 128^2$, so $TM \leq 7200$ complex measurements describe 16384 unknowns and $\mathbf{A}$ has a non-trivial null space. For a volume with slices $\boldsymbol{f}^{(z)}$, $z = 1, \ldots, Z$, the slice-wise model is $\boldsymbol{y}^{(z)} = \mathbf{A}\boldsymbol{f}^{(z)} + \boldsymbol{n}^{(z)}$.

6.3 Objective for scattering

With $\boldsymbol{f}_\Theta$ the field (6.1) sampled on the grid,

$$\mathcal{L}(\Theta) = \frac{\sum_z \left\| \mathbf{A}\boldsymbol{f}_\Theta^{(z)} - \boldsymbol{y}^{(z)} \right\|_2^2}{\sum_z \left\| \boldsymbol{y}^{(z)} \right\|_2^2} \; + \; \lambda \, \frac{1}{|\Omega|} \sum_{\boldsymbol{x} \in \Omega} f_\Theta(\boldsymbol{x}), \tag{6.3}$$

where the second term equals $\lambda \left\| \boldsymbol{f}_\Theta \right\|_1 / |\Omega|$ because $f_\Theta \geq 0$. The baseline solves the same problem over the grid values directly, $\min_{\boldsymbol{f} \in [0,1]^{|\Omega|}} \left\| \mathbf{A}\boldsymbol{f} - \boldsymbol{y} \right\|_2^2 / \left\| \boldsymbol{y} \right\|_2^2 + \lambda \, \overline{\boldsymbol{f}}$, by projected Adam from $\boldsymbol{f} = \mathbf{0}$.

Why grids do not grow ghosts. The gradient of the data term with respect to $\boldsymbol{f}$ is $2 \operatorname{Re} \mathbf{A}^{\mathsf{H}}(\mathbf{A}\boldsymbol{f} - \boldsymbol{y}) / \left\| \boldsymbol{y} \right\|^2$, which lies in the range of $\mathbf{A}^{\mathsf{H}}$, the orthogonal complement of the null space. Gradient steps from zero therefore never add null-space components (up to the effect of the projection onto $[0, 1]$). Gaussians break this property: a Gaussian inserted anywhere carries mass that may lie largely in the null space, where the data term exerts no force to remove it. The sparsity term in (6.3) supplies that force.

6.4 Adaptive density control

Every $K = 250$ iterations while $k \leq 0.6 \, K_{\max}$ the back-projected residual

$$\boldsymbol{\rho} = \operatorname{Re}\big(\mathbf{A}^{\mathsf{H}}(\boldsymbol{y} - \mathbf{A}\boldsymbol{f}_\Theta) \big) \tag{6.4}$$

is computed, n_{add} new Gaussians are created at the grid points with the largest $|\rho|$ (scale 1.5 cells, amplitude proportional to $|\rho|$), and Gaussians with $a_i < 10^{-3}$ are removed.

6.5 4D representation

For a sequence $\omega_0, \ldots, \omega_{T-1}$ observed through masks $\mathbf{M}_t$, canonical parameters θ_i are deformed by a network D_ϕ:

$$\theta_i(t) = \theta_i + \mathbf{S} \, D_\phi\big(\gamma(\boldsymbol{\mu}_i / L), \, \gamma(t/T) \big), \qquad \gamma(p) = \big[\sin(2^k \pi p), \cos(2^k \pi p) \big]_{k=0}^{K-1}, \tag{6.5}$$

where $\mathbf{S}$ scales position offsets to pixels and γ are Fourier features [23]. The training loss over a random batch $\mathcal{B}$ of observed frames is

$$\mathcal{L}(\Theta, \phi) = \sum_{t \in \mathcal{B}} \frac{\left\| \mathbf{M}_t \odot (f_{\Theta(t)} - \tilde{\omega}_t) \right\|_2^2}{\left\| \mathbf{M}_t \right\|_1} \; + \; \lambda_T \sum_{t \in \mathcal{B}} \frac{1}{N} \sum_i \left\| D_\phi(\cdot, t+1) - D_\phi(\cdot, t) \right\|_2^2, \tag{6.6}$$

with $\tilde{\omega}_t$ the (normalised) observations and $\lambda_T \geq 0$ the temporal smoothness weight.

6.6 Mamba deformation

For Gaussian i, the inputs $\boldsymbol{x}_t$ ($t = 0, \ldots, T-1$) are linear embeddings of $[\gamma(\boldsymbol{\mu}_i), \gamma(t)]$. A Mamba block [7] computes, after a gated input projection and a short causal convolution,

$$\Delta_t = \mathrm{softplus}(\mathbf{W}_\Delta \boldsymbol{x}_t), \qquad \mathbf{B}_t = \mathbf{W}_B \boldsymbol{x}_t, \qquad \mathbf{C}_t = \mathbf{W}_C \boldsymbol{x}_t, \tag{6.7}$$

$$\boldsymbol{h}_t = \exp(\Delta_t \mathbf{A}) \odot \boldsymbol{h}_{t-1} + \Delta_t \, \mathbf{B}_t \, \boldsymbol{x}_t, \qquad \boldsymbol{y}_t = \mathbf{C}_t \boldsymbol{h}_t + \mathbf{D} \boldsymbol{x}_t, \tag{6.8}$$

with a diagonal state matrix $\mathbf{A} = -\exp(\mathbf{A}_{\log})$, followed by multiplicative gating with $\mathrm{SiLU}(\boldsymbol{z}_t)$ and a residual connection. Because $\Delta_t, \mathbf{B}_t, \mathbf{C}_t$ depend on the input, the model selects what to remember along the trajectory. The bidirectional variant adds a second stack applied to the time-reversed sequence and sums the two outputs.

6.7 Evaluation metrics

For a reconstruction $\hat{f}$ and ground truth f with data range R,

$$\mathrm{PSNR} = 10 \log_{10} \frac{R^2}{\frac{1}{|\Omega|} \left\| \hat{f} - f \right\|_2^2}, \qquad \mathrm{relL2} = \frac{\left\| \hat{f} - f \right\|_2}{\|f\|_2}, \tag{6.9}$$

and SSIM [25] computed with the default 7×7 uniform window of scikit-image ($R = 1$ for permittivity, $R = 2$ for normalised vorticity). For 4D, metrics are averaged separately over observed (training) and held-out frames.

Chapter 7

Experimental Setup

7.1 Data

- **2D cells (scattering).** The first samples of the InverseBench `inv-scatter-test` set: 128×128 permittivity images of 1–6 HL60 nuclei simulated with CytoPacq [26], values in $[0, 1]$.
- **3D cells (scattering).** (a) Synthetic HL60-like nuclei generated by our code: 1–6 soft, rotated ellipsoids with smooth internal texture in a $32 \times 128 \times 128$ volume. (b) Real CytoPacq nuclei from BBBC024 of the Broad Bioimage Benchmark Collection [14] (ground-truth label masks, 20 nuclei per volume), converted to foreground masks, cropped to the central half in x and y and resampled to $32 \times 128 \times 128$.
- **Fluid (4D).** Initial vorticity fields from the InverseBench `navier-stokes-test` set ($Re = 200$), rolled forward with the InverseBench pseudo-spectral solver to 21 frames spaced $\Delta t = 0.05$ apart, divided by 10 to lie in about $[-1, 1]$. Frames 2, 6, 10, 14 and 18 are held out; the rest are observed completely or at a random 10% of pixels per frame.

7.2 Forward models and noise

Scattering uses the InverseBench configuration: domain $0.18 \times 0.18\,\mathrm{m}$, wavelength six grid cells, 20 transmitters and 360, 180 or 60 receivers on a circle of radius 1.6 m, complex Gaussian noise $\sigma = 10^{-4}$ with a fixed seed per sample so that Gaussian and baseline fits see identical measurements.

Table 7.1: Default settings (overridable from the command line; every run stores its exact settings).

Setting	Scattering 2D / 3D	Fluid 4D
Iterations	3000	3000
Optimiser, learning rate	Adam, 0.05 (Gaussians), 0.01 (grid)	Adam, 0.01 (Gaussians), 10^{-3} (network)
Schedule	cosine	cosine
Max. / initial Gaussians	2000 / 250 (2D); 8000 / 2000 (3D)	2000 / 2000
Window half-size h	8 (2D), 6 (3D)	10, periodic
Densification	every 250 it., until 60%, $+n_{\text{init}}/4$	none
Sparsity λ	0, 0.1, 0.3, 1, 3	–
Deformation	–	MLP (3×128), Mamba (2 blocks, $d = 64$, state 16)
Batch	one sample	4 frames
Samples per setting	3 (5 in the final runs)	3 sequences

7.3 Hyper-parameters

7.4 Hardware and software

All experiments ran on a single NVIDIA Tesla T4 (16 GB) on Google Colab or Kaggle, with Python 3, PyTorch, NumPy, SciPy and scikit-image. A 2D scattering fit takes about 13 s; a 3D fit a few minutes; a 4D fit with the MLP about 3 min and with Mamba about 15 min (sequential scan).

7.5 Protocol

For each sample, receiver count and method, measurements are generated once and both the Gaussian and the grid method are fitted to them. Reported numbers are means over samples; per-sample values, figures and run configurations are stored in the repository (`results/`).

Chapter 8

Initial Baseline Results

All numbers are means over three samples (PSNR in dB / SSIM) unless stated otherwise. The Gaussian and grid methods use identical measurements and data terms.

8.1 Representation check

Fitting Gaussians directly to the ground truth (no physics) confirms that the representation can express the fields: 48.2 dB / 0.998 on the 2D cells with about 680 Gaussians, and 65.2 dB / 0.999 on the synthetic 3D volumes with 5500 Gaussians.

8.2 2D inverse scattering on InverseBench cells

Table 8.1 lists the reconstructions from measurements only. Without regularisation, Gaussians are 6–7 dB below the pixel grid with 360 or 180 receivers and 2.7 dB below with 60. Increasing the Gaussian budget did *not* help: with aggressive densification (about 2000 Gaussians) PSNR fell to 22.9 / 22.6 / 19.6 dB. The reconstructions show why (Figure 8.1): a faint copy of the cell appears outside the object. These "ghosts" sit where the back-projected residual is large but the measurements carry almost no information, so the data term cannot remove them (Chapter 6). Adding the sparsity term removes the ghosts. With $\lambda = 0.3$, Gaussians improve at every receiver count, and at 60 receivers they exceed the pixel grid in SSIM (0.940 vs. 0.878) while staying 1 dB lower in PSNR. Larger weights ($\lambda = 1, 3$) shrink the cells and lower both metrics, so the weight must be tuned.

8.3 3D inverse scattering

On synthetic 3D nuclei (Table 8.2, Figure 8.2) Gaussians are almost unaffected by the number of receivers (40.6–40.9 dB), while the voxel grid degrades from 44.1 to 37.3 dB and becomes speckled. With 60 receivers Gaussians are better on all three volumes (+3.6 dB

Table 8.1: 2D inverse scattering, InverseBench cells (PSNR dB / SSIM, mean of 3).

Method	360 receivers	180 receivers	60 receivers
Gaussians	30.7 / 0.959	30.4 / 0.957	26.5 / 0.814
Gaussians, ∼2000 (aggressive densification)	22.9 / –	22.6 / –	19.6 / –
Gaussians + sparsity, $\lambda = 0.3$	31.4 / 0.983	31.0 / **0.982**	28.2 / **0.940**
Gaussians + sparsity, $\lambda = 1$	27.6 / 0.958	27.6 / 0.958	25.2 / 0.908
Gaussians + sparsity, $\lambda = 3$	21.6 / 0.875	21.6 / 0.876	20.5 / 0.826
Pixel grid	**36.7** / **0.985**	**35.3** / 0.976	**29.2** / 0.878

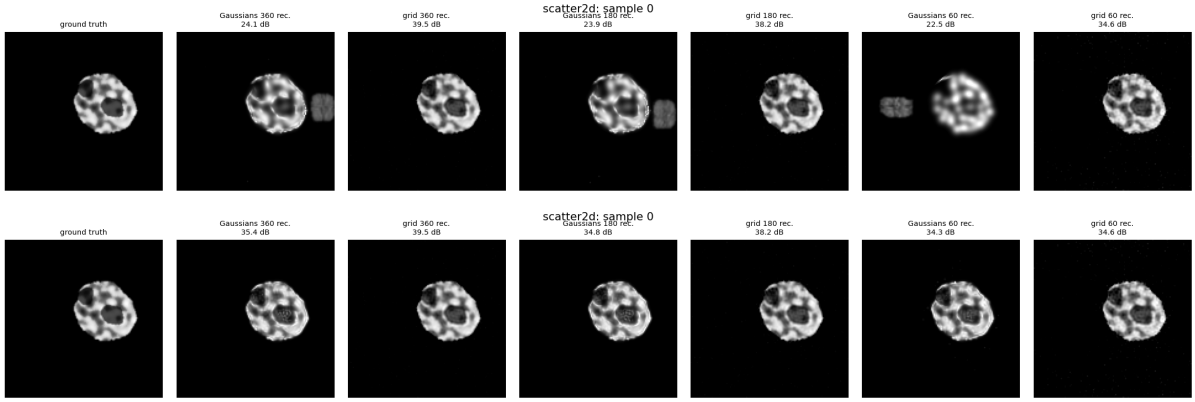


Figure 8.1: Cell 0. Top: aggressive densification without regularisation produces a ghost copy beside the cell. Bottom: with the sparsity term ($\lambda = 0.3$) the ghost is gone. Columns: ground truth, then Gaussian and pixel reconstructions at 360, 180 and 60 receivers.

on average) and have higher SSIM at every receiver count. Because these volumes consist of smooth ellipsoids, which favour Gaussians, the experiment is being repeated on real CytoPacq nuclei from BBBC024 with and without sparsity (final runs, Table 8.2).

Table 8.2: 3D inverse scattering, $32 \times 128 \times 128$ volumes (PSNR dB / SSIM, mean of 3).

Volumes	Method	360 receivers	180 receivers	60 receivers
Synthetic nuclei	Gaussians	40.8 / **0.994**	40.6 / **0.994**	**40.9** / **0.988**
	Voxel grid	**44.1** / 0.987	**41.5** / 0.977	37.3 / 0.932
BBBC024 (CytoPacq)	Gaussians (+ sparsity)	[final run]	[final run]	[final run]
	Voxel grid	[final run]	[final run]	[final run]

8.4 4D Gaussians for Navier–Stokes

Table 8.3 reports PSNR on observed (training) and held-out frames. Static Gaussians reach only about 17 dB: the flow changes too much over 21 frames for one fixed field. A deformation network adds about 10 dB on held-out frames. Mamba fits the observed frames much better than the MLP (36.1 vs. 30.3 dB with full observations), but on held-out frames the two are within 0.3 dB, i.e. Mamba's extra capacity is spent memorising

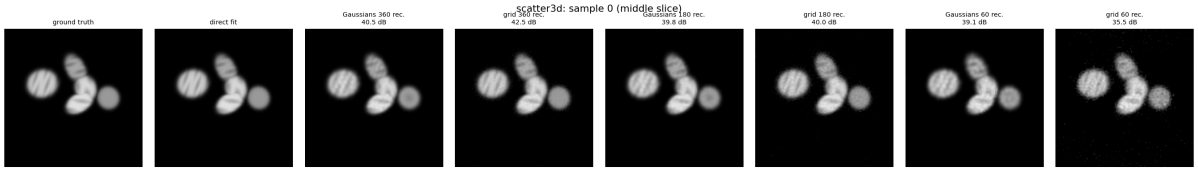


Figure 8.2: Middle slice of synthetic volume 0: ground truth, direct fit, then Gaussian and voxel reconstructions at 360, 180 and 60 receivers.

observed times rather than modelling the motion between them. With 10% observations the deformation models lose about 4.5 dB, and the MLP shows streak artefacts in early frames (Figure 8.3). Mamba is about five times slower because of its sequential scan. The final runs add temporal smoothness and a bidirectional Mamba to target the generalisation gap.

Table 8.3: 4D Navier–Stokes, PSNR (dB) on held-out frames; observed-frame PSNR in brackets (mean of 3 sequences).

Observed pixels	Static	MLP (4DGS-style)	Mamba
100%	17.0 (16.8)	27.6 (30.3)	**27.9** (**36.1**)
10%	16.0 (15.9)	**23.2** (24.0)	23.1 (**27.7**)
Training time per sequence	3 min	3 min	15 min

8.5 Answers to the research questions so far

RQ1 Yes on smooth 3D volumes, where Gaussians are more robust than voxels to scarce measurements; on textured 2D cells the pixel grid keeps higher PSNR, while regularised Gaussians win in SSIM with 60 receivers.

RQ2 Adaptive densification can populate the operator's null space with ghost structures; a sparsity term on total intensity prevents them.

RQ3 Yes: canonical Gaussians plus a deformation network reconstruct the flow at held-out times, about 10 dB better than static Gaussians, from full or 10% observations.

RQ4 Not yet: causal Mamba matches the MLP on held-out frames and overfits observed ones; regularised and bidirectional variants are under evaluation.

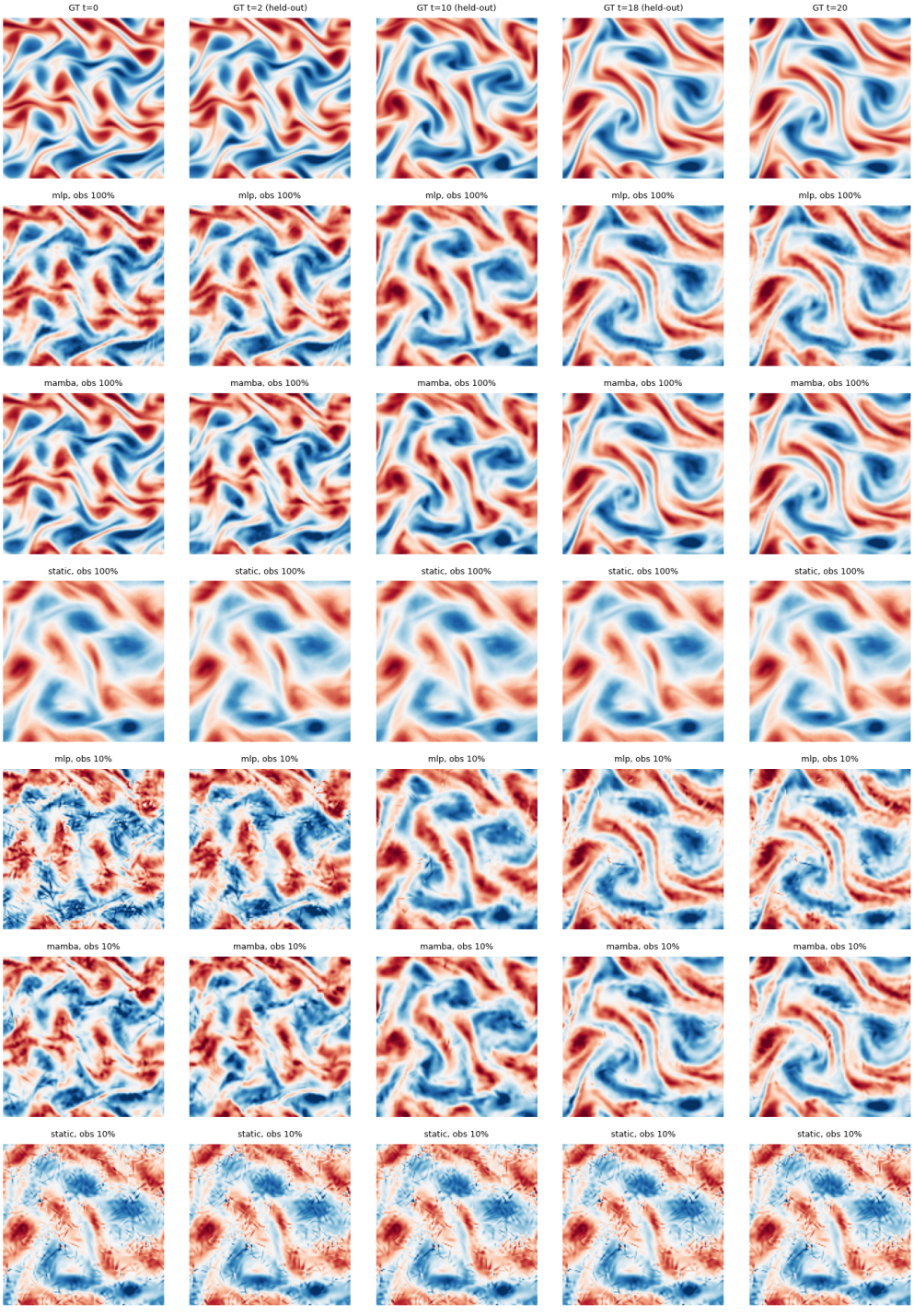


Figure 8.3: Vorticity of sequence 0 at five times (held-out frames marked). Rows: ground truth, then MLP, Mamba and static models with 100% and 10% of pixels observed.

Chapter 9

Discussion, Limitations and Plan for Thesis II

9.1 Discussion

Replacing the camera model of Gaussian splatting with a physics operator works: the same representation, optimiser and density control recover cells from scattered fields and moving vorticity from sparse samples. The representation acts as a prior that is helpful when the object matches it (smooth nuclei, scarce data) and neutral or harmful when it does not (fine texture, abundant data). The most instructive finding is the ghost failure mode: adaptivity, the main strength of Gaussian splatting with cameras, becomes a liability when the measurements are blind to part of the domain, and must be balanced by an explicit prior. For dynamics, the deformation model matters far more than the choice of network, and a sequence model must be regularised to turn its capacity into better interpolation in time.

9.2 Limitations

- The 3D physics is slice-wise 2D scattering, not a true 3D Born model with sensors around the volume.
- Results use three to five samples per setting; differences of a few tenths of a dB are not yet significant.
- The 4D problem reconstructs a field from observations of itself; InverseBench's harder task of recovering the initial condition through the solver is not addressed.
- No learned prior is used, and no comparison with InverseBench's diffusion methods is made (by design for Thesis I).

- Hyper-parameters (sparsity weight, Gaussian budget, window size) were tuned on few samples.

9.3 Plan for Thesis II

1. A true 3D scattering operator (3D Green's function, transmitters and receivers on a sphere) and, if time allows, multiple scattering.
2. Systematic evaluation on the full InverseBench test sets with confidence intervals, and comparison with the benchmark's diffusion-prior methods.
3. Physics-aware density control (densify only within the support indicated by the data; null-space-aware regularisers) instead of a global sparsity term.
4. Motion models: temporal regularisation, bidirectional and hardware-efficient Mamba, and advection-informed deformation that uses the velocity field implied by the vorticity.
5. The Navier–Stokes initial-value inverse problem through a differentiable solver.
6. Connection to the original direction of this work: attaching semantic features to physical Gaussians, in the spirit of LangSplat [19] and 4D LangSplat [12].

9.4 Conclusion

This Thesis I establishes a working, reproducible proof of concept for 3D and 4D Gaussian representations in two scientific inverse problems, identifies when they help and how they fail, and defines the experiments that Thesis II will scale up.

Bibliography

[1] Max Born and Emil Wolf. *Principles of Optics*. Cambridge University Press, 7th edition, 1999.

[2] Yuanhao Cai et al. Radiative gaussian splatting for efficient X-ray novel view synthesis. In *European Conference on Computer Vision (ECCV)*, 2024.

[3] Ang Cao and Justin Johnson. HexPlane: A fast representation for dynamic scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[4] Hyungjin Chung, Jeongsol Kim, Michael T. Mccann, Marc L. Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] Chengyuan Deng, Shihang Feng, Hanchen Wang, Xitong Zhang, Peng Jin, Yinan Feng, Qili Zeng, Yinpeng Chen, and Youzuo Lin. OpenFWI: Large-scale multi-structural benchmark datasets for full waveform inversion. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[6] Sara Fridovich-Keil, Giacomo Meanti, Frederik Rahbæk Warburg, Benjamin Recht, and Angjoo Kanazawa. K-planes: Explicit radiance fields in space, time, and appearance. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[7] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[8] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.

[9] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[11] Aviad Levis, Pratul P. Srinivasan, Andrew A. Chael, Ren Ng, and Katherine L. Bouman. Gravitationally lensed black hole emission tomography. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[12] Wanhua Li et al. 4D LangSplat: 4D language gaussian splatting via multimodal large language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[13] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[14] Vebjorn Ljosa, Katherine L. Sokolnicki, and Anne E. Carpenter. Annotated high-throughput microscopy image sets for validation. *Nature Methods*, 9(7):637, 2012.

[15] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3D gaussians: Tracking by persistent dynamic view synthesis. In *International Conference on 3D Vision (3DV)*, 2024.

[16] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.

[17] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4), 2022.

[18] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[19] Minghan Qin, Wanhua Li, Jiawei Zhou, Haoqian Wang, and Hanspeter Pfister. LangSplat: 3D language gaussian splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[20] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[21] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60(1–4):259–268, 1992.

[22] Yu Sun, Zhihao Xia, and Ulugbek S. Kamilov. Efficient and accurate inversion of multiple scattering with deep learning. *Optics Express*, 26(11), 2018.

[23] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[24] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Deep image prior. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[25] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[26] David Wiesner, David Svoboda, Martin Maška, and Michal Kozubek. CytoPacq: A web-interface for simulating multi-dimensional cell imaging. *Bioinformatics*, 35(21), 2019.

[27] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D gaussian splatting for real-time dynamic scene rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[28] Zeyu Yang, Hongye Yang, Zijie Pan, and Li Zhang. Real-time photorealistic dynamic scene representation and rendering with 4D gaussian splatting. In *International Conference on Learning Representations (ICLR)*, 2024.

[29] Jure Zbontar et al. fastMRI: An open dataset and benchmarks for accelerated MRI. *arXiv preprint arXiv:1811.08839*, 2018.

[30] Ruyi Zha et al. R^2-Gaussian: Rectifying radiative gaussian splatting for tomographic reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[31] Xinjie Zhang et al. GaussianImage: 1000 FPS image representation and compression by 2D gaussian splatting. In *European Conference on Computer Vision (ECCV)*, 2024.

[32] Hongkai Zheng, Wenda Chu, Bingliang Zhang, Zihui Wu, Austin Wang, Berthy T. Feng, Caifeng Zou, Yu Sun, Nikola Kovachki, Zachary E. Ross, Katherine L. Bouman,

and Yisong Yue. InverseBench: Benchmarking plug-and-play diffusion priors for inverse problems in physical sciences. In *International Conference on Learning Representations (ICLR)*, 2025.

[33] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *International Conference on Machine Learning (ICML)*, 2024.

Appendix A

Reproducibility

The code, configurations and results are in the thesis repository (`src/thesis_gs` package, `bin/` scripts, `configs/`, `notebooks/`, `results/`, `tests/`). InverseBench and its test data are downloaded automatically; BBBC024 is downloaded by the Kaggle notebook.

```
# install, smoke test, 2D run, final runs, unit tests
pip install -r requirements.txt
python bin/thesis-gs --task all --quick --out outputs/smoke
python bin/thesis-gs --task scatter2d --l1 0.3 --out outputs/s2d
python bin/run_experiments.py configs/final_part_a.json \
       --out outputs/final_a --volumes /path/to/BBBC024
python bin/run_experiments.py configs/final_part_b.json \
       --out outputs/final_b
pytest -q
```

Every run writes `run_config.json` with its exact settings, `results.csv` with per-sample metrics, `summary.csv` with means, and `reconstructions.png`.